

Detecting Anomalies in Encrypted Traffic via Deep Dictionary Learning

Junchi Xing*, Chunming Wu^{†*§}

^{*}College of Computer Science and Technology, Zhejiang University

[†]Zhejiang Lab

Email: {jcxing, wuchunming}@zju.edu.cn

Abstract—The widely used encryption of network traffic poses a great challenge to anomaly detection. Currently, the supervised and semi-supervised solutions suffer from the problems of noisy label of data, non-stationary traffic distribution, and huge resource consumption for offline training. To address this, in this paper, we present an unsupervised, robust, and online anomaly detection method for encrypted traffic by using deep dictionary learning, called D2LAD. D2LAD offers the potential to extract sequential features from raw encrypted traffic data with the help of an LSTM-based autoencoder. Then, according to the sequential features, D2LAD is able to explore the hidden normal patterns by iterative deep dictionary learning. Eventually, D2LAD can obtain the anomaly score of raw data by calculating its relevance to the deep dictionary. We implement a prototype of D2LAD and evaluate its effectiveness and performance by experiments using realistic datasets. The experimental results show that our method achieves high accuracy and low resource usage, and outperforms the representative state-of-the-art methods in real-world settings.

Index Terms—Anomaly Detection, Deep Dictionary Learning, Encrypted Traffic, LSTM-based Autoencoder

I. INTRODUCTION

Anomaly detection is a crucial task in network security, which identifies malicious and unexpected activities from network traffic [1], [2]. Nowadays, for privacy concerns, the percentage of network traffic that makes use of encryption has been rapidly increasing [3], [4]. However, traditional pattern-matching detection methods like Deep Packet Inspection that rely on plaintext traffic [5]–[7] can be foiled. Because they cannot be applied to the randomized communication contents after encryption [8]. Therefore, anomaly detection for encrypted traffic becomes an important problem and attracts widespread attention.

Currently, the mainstream solutions are supervised [8]–[13] or semi-supervised [14] machine learning methods based on the statistical [8]–[12], [14] or sequential [13] features that are not affected by encryption. Unfortunately, these solutions are difficult to materialize as viable methods for real-world anomaly detection due to three primary reasons. **Firstly**, labeled normal samples inevitably include noise, and labeled anomalous traffic samples are rarely available [15]. **Secondly**, the underlying traffic distribution is non-stationary [8], consequently normality may vary over time. **Thirdly**, the size of the sample set is typically enormous, training at once implies huge memory usage and time consumption.

To account for the above problem, in this paper, we propose Deep Dictionary Learning based Anomaly Detection, named D2LAD, which is an *unsupervised*, *robust*, and *online* detection method for encrypted traffic. The idea behind D2LAD is to (1) extract sequential features from raw encrypted traffic data, (2) learn the normal traffic pattern of traffic by means of a deep dictionary [16], and (3) detect the anomalies according to relevance of the traffic data and the deep dictionary. To achieve this, (1) we input the raw traffic to a pre-trained LSTM(Long Short-Term Memory)-based autoencoder [17], [18] and output the hidden layer vectors as the sequential features; (2) based on the extracted sequential features, we construct a deep dictionary to represent the normal traffic patterns and use a lay-wise solution to incrementally update the dictionary; and (3) we obtain the relevance of traffic data and deep dictionary by computing the dictionary feature rarity and coefficient.

We implement a prototype of D2LAD using Keras [19] library. We perform experiments for evaluating the effectiveness and performance of D2LAD using real-world encrypted traffic dataset. The experimental results show that our unsupervised method achieves comparable accuracy (F_1 score $\approx 94.5\%$) with online data processing and is more robust to noisy label problem and non-stationary traffic distribution.

Contributions. This paper makes the following main contributions:

- We propose a novel method that can incrementally learn the normal patterns of raw encrypted traffic using a deep dictionary.
- We introduce the construction and layer-wise updating of the deep dictionary.
- We definite the anomaly situation by the relevance of traffic data and deep dictionary.
- The experiment results demonstrate that our method achieves an accuracy of 94.5% and outperforms the state-of-the-art methods in real-world settings.

The remainder of this paper is organized as follows. Section II presents the preliminaries. Section III presents the design of the proposed D2LAD in detail. Section IV evaluates the performance of D2LAD. Section V discusses on related work. Section VI concludes the paper.

II. PRELIMINARIES

In this section, we firstly state the problem of encrypted traffic anomaly detection. Sequentially, we introduce LSTM-

Corresponding Author[§].

based autoencoder and deep dictionary learning, and explain how they take effect in our method for addressing the above problem.

A. Problem Statement

The problem in this paper is to detect anomalies in encrypted traffic with the following constraints: (1) Due to privacy concerns, we never decrypt the traffic, and only use the metadata of encrypted traffic flows to mine the anomalies. The traffic can be encrypted using common-used protocols such as TLS or SSL. (2) We aim to address the problem in an unsupervised fashion without empirical labeled samples. (3) An online detection with limited memory usage is needed for real-world scenarios.

B. LSTM-based Autoencoder

LSTM-based autoencoder is able to deeply extract implicit and important features from sequential data without vanishing gradient problem [17], [18]. Specifically, it learns the features by minimizing the reconstruction errors between the raw data and reconstructed data in an unsupervised manner. LSTM-based autoencoder is recently proven to be effective in many areas like sentiment analysis [20] and face recognition [21]. Therefore, we consider using it to extract sequential features from the metadata of encrypted traffic flows.

C. Deep Dictionary Learning

Deep dictionary learning has the potential to incrementally learn a sparse dictionary that can represent the data [22] (viz., Fig. 1). The relationship between dictionary $\mathbf{D}_n$ and data X can be expressed as: $X = \Gamma_1 \varphi(\Gamma_2 \varphi(\dots \varphi(\Gamma_n \mathbf{D}_n)))$, where n is the layer number, Γ is coefficients and φ is a non-linear activation. The multi-level structure of deep dictionary learning makes it more accurate than traditional dictionary learning [16]. Hence, in this paper, we use deep dictionary learning to abstract the normal patterns with reference to the sequential features of encrypted traffic and, subsequently, to detect anomalies.

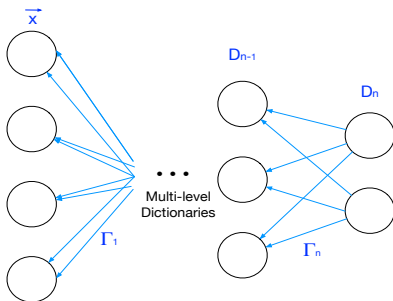


Fig. 1. Deep dictionary learning architecture.

III. METHOD

A. Overview

The overview of D2LAD framework is sketched in Fig. 2, which can be divided into four steps. We firstly preprocess

the input raw traffic data for boiling away needless traffic information. Secondly, the sequential features will be extracted using a pre-trained LSTM-based autoencoder. Then, the deep dictionary that learns the normal traffic pattern is updated asymptotically. Finally, the anomalous traffic can be detected based on its relevance to the deep dictionary. In the following sections, we will introduce the four steps, sequentially.

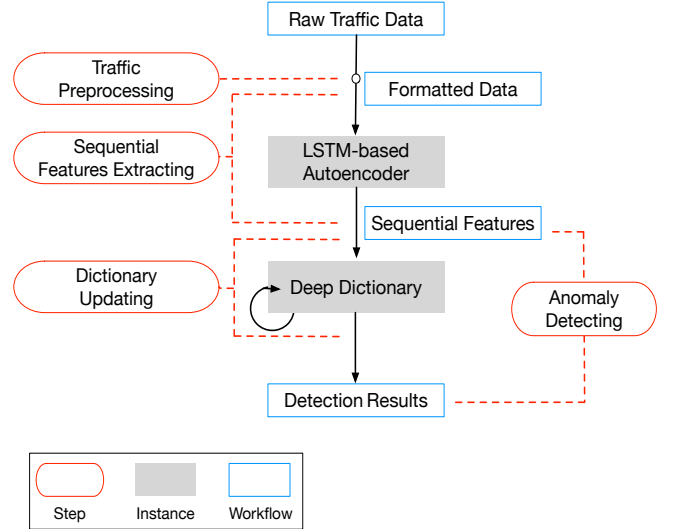


Fig. 2. Overview of D2LAD.

B. Traffic Preprocessing

We seek to collect representative traffic information from high-dimensional raw traffic for detection and format it. Let $\vec{p}$ be the attribute vector of a packet, and the attributes can fall into three categories. (1) TLS handshake parameters (e.g., type and number of cipher suites, length of public-key) [10]; (2) packet length; and (3) interval of packets [13]. The categorical attributes are encoded by one-hot method, and the numerical attributes will be normalized. In this case, the dimension of the attributes can be easily reduced by the next applied LSTM-based Autoencoder. Then, we use $f = [\vec{p}_1, \vec{p}_2, \dots, \vec{p}_m]$ as the formatted data for further detection, where each $\vec{p}$ is arranged in chronological order, and $m \in \mathbf{Z}^+$ is the length of the packet sequence and may vary.

C. Sequential Features Extracting

In this section, we use a pre-trained LSTM-based autoencoder to extract sequential features for each f in $\mathbf{F}$. We will briefly describe the structure of the LSTM-based autoencoder along with its training process. After the training, the hidden layer would represent sequential features of the input data [20].

LSTM-based autoencoder has two parts: encoder and decoder. The encoder compresses $f = [\vec{p}_1, \vec{p}_2, \dots, \vec{p}_m]$ as hidden state vectors based on generic LSTM [23]:

$$h_i = \kappa_\phi(\vec{p}_i, c_{i-1}), \quad (1)$$

where c is the memory cell, and $\kappa_\phi(\cdot)$ is the encoding function with the parameter set of the encoder part (ϕ). Then, we pass h_i to the decoder part which reconstructs f through:

$$\hat{h}_i = \kappa_\psi(h_i, \hat{h}_{i-1}, \hat{c}_{i-1}), \quad (2)$$

$$\hat{p}_i = \rho(\hat{h}_i), \quad (3)$$

where $\hat{h}_i$ and $\hat{p}_i$ are the reconstructed hidden vectors and input, $\kappa_\psi(\cdot)$ is the encoding function with the parameter set of the encoder part (ψ), and $\rho(\cdot)$ is commonly set to $\tanh(\cdot)$ and apply pointwise to vectors.

The training process is implemented by modifying ϕ and ψ in hope of minimizing the mean-square error (MSE) between $\hat{p}_i$ and $\tilde{p}_i$. Moreover, this process is performed in advance using the background traffic data set (this will be described in Section IV). Therefore, this model can directly and quickly extract features in online detection.

D. Dictionary Learning

Here, the target for anomaly detection becomes the extracted sequential features. Our idea is to use a sparse deep dictionary to learn the feature patterns. And as data volumes grow, the dictionary gradually tends to represent the normal features' pattern. In this section, we will introduce the learning process which is the foundation anomaly detection.

Dictionary construction. A new sequential feature vector is regarded as input data to the dictionary, which is denoted as $\vec{x}$. A deep dictionary $\mathbf{D}$ consists of $n-1$ representing layers ($\mathbf{D}_1$ to $\mathbf{D}_{n-1}$) and a dictionary layer ($\mathbf{D}_n$) to represent the data in succession. Each layer has a limited number of feature vectors. The relationship between dictionary $\mathbf{D}_n$ and data $\vec{x}$ can be expressed as:

$$X = \Gamma_1 \varphi(\Gamma_2 \varphi(\dots \varphi(\Gamma_n \mathbf{D}_n))), \quad (4)$$

, where Γ is coefficients and φ is a non-linear activation. The dictionary learning task is to train a dictionary such that it is well adapted for reconstructing a set of data points. At any time t , the dictionary draws one input $\vec{x}$, the dictionary with a sparsity constraint can be learned by solving the following optimization problem:

$$\min_{\Gamma, \mathbf{D}} \|\vec{x} - \Gamma_1 \varphi(\Gamma_2 \varphi(\dots \varphi(\Gamma_n \mathbf{D}_n)))\|_F^2 + \mu \|\mathbf{D}_n\|_1, \quad (5)$$

where μ is the sparse parameter to balance representing quality and computation cost.

Dictionary updating. The optimal dictionary is the solution to the above problem. However, such a problem is highly non-convex and requires solving a big number of parameters. With the limited amount of data, it will cause overfitting. Thus, we adopt a layer-wise updating in a greedy manner.

(1) *First layer updating.* Firstly we update the parameters of the first representing layer with the following objective function:

$$\min_{\Gamma_1, \mathbf{D}_1} \|\vec{x} - \Gamma_1 \mathbf{D}_1\|_F^2. \quad (6)$$

This can be solved by using alternating minimization, whose optimality has been proven in [24]. Therefore, the dictionary of the first layer $\mathbf{D}_1$ and its coefficients Γ_1 can be computed by:

$$\mathbf{D}_1 \leftarrow \min_{\mathbf{D}_1} \|\vec{x} - \Gamma_1 \mathbf{D}_1\|_F^2, \quad (7a)$$

$$\Gamma_1 \leftarrow \min_{\Gamma_1} \|\vec{x} - \Gamma_1 \mathbf{D}_1\|_F^2. \quad (7b)$$

Then both (7a) and (7b) can be easily solved by least square method.

(2) *Middle layers updating.* Based on (4), we can obtain that $\mathbf{D}_{i-1} = \Gamma_i \mathbf{D}_i$ ($2 \leq i \leq n-1$), thus these layers can be learned by continuously solving the objective function:

$$\min_{\Gamma_i, \mathbf{D}_i} \|\varphi^{-1}(\mathbf{D}_{i-1}) - \Gamma_i \mathbf{D}_i\|_F^2. \quad (8)$$

(3) *Dictionary/last layer updating.* Considering that this layer learning sparse features, the updating process can be express as:

$$\mathbf{D}_n \leftarrow \min_{\mathbf{D}_n} \|\varphi^{-1}(\mathbf{D}_{n-1}) - \Gamma_n \mathbf{D}_n\|_F^2 + \mu \|\mathbf{D}_n\|_1, \quad (9a)$$

$$\Gamma_n \leftarrow \min_{\Gamma_n} \|\varphi^{-1}(\mathbf{D}_{n-1}) - \Gamma_n \mathbf{D}_n\|_F^2, \quad (9b)$$

where (9a) can be solved by using the Iterative Soft Thresholding Algorithm (ISTA) [25], and (9b) can be solved through least square method as before.

E. Anomaly Detecting

After the updating of the dictionary based a new data vector, our task is to identify whether it is anomalous. To this end, given the dictionary which represents the normal pattern, we use the correlation between data and dictionary as a metric to quantify the data anomaly. We argue that such correlation depends jointly on two factors: (1) feature rarity: the frequency of each feature in the dictionary in reconstructing the data, and (2) feature coefficient: the strength by which it is used in reconstructing the data. It is based on the assumption that the representation of anomalous data more depends on rare feature.

First of all, we consider using rarity score θ to measure feature rarity. In this case, we will use entropy gain to compute the rarity of feature [26]. To compute the entropy gain, we firstly compute the instantaneous activity ratio $p^j(t)$ of j^{th} feature at time t as:

$$p^j(t) = \frac{\sum_{h=1}^{|\mathbf{D}_{n-1}|} |\Gamma_n^{h,j}(t)|}{\sum_{i=1}^k \sum_{h=1}^{|\mathbf{D}_{n-1}|} |\Gamma_n^{h,i}(t)|}, \quad (10)$$

where $\Gamma_n^{h,j}(t)$ indicates the coefficients between the h^{th} vector in $\mathbf{D}_{n-1}$ and the j^{th} vector in $\mathbf{D}_n$ at time t .

Then, the summary activity ratio $q(t)$ can be incrementally updated using EWMA(Exponential Weighted Moving Average) as follows:

$$q(t) = (1 - \alpha)q(t-1) + \alpha p(t). \quad (11)$$

Next, we are able to obtain the entropy gain using:

$$EG(q_j) = \frac{\partial H(\vec{q})}{\partial q_j} = -H(\vec{q}) - q_j - \log q_j - q_j \log q_j. \quad (12)$$

And if $EG(q_j) < 0$, we let $EG(q_j) = 0$ in order to overlook the frequent features. Hence, the rarity score of the j^{th} feature can be computed as:

$$\theta_j = \frac{EG(q_j)}{\sum_{i=1}^k EG(q_i)}. \quad (13)$$

Moreover, the feature coefficient can be easily obtained from $\Gamma_n = [\gamma_1, \dots, \gamma_{|\mathbf{D}_n-1|}]$. Therefore, the correlation between the data and the dictionary as well as the anomaly score ζ can be defined as:

$$\zeta(\vec{x}) = |\vec{\theta}|^T |\Gamma_n|, \quad (14)$$

and if the anomaly score of data exceed an assigned threshold value τ , we consider that anomalies occur in encrypted traffic.

IV. EVALUATION

In this section, we present the datasets, experiment setup, and perform experiments to evaluate the effectiveness and performance of D2LAD.

A. Dataset

Malware traffic dataset. It includes a) 251,586 malware flows for CTU-13 dataset; b) 17,442 flows from malware-traffic-analysis.net dataset [27]; c) 507,175 flows from Stratosphere IPS [28]; d) 2,998 flows from Lastline [29]. These datasets can be accessed on Google Drive ¹.

Benign traffic dataset. It is composed of 1,437,189 background and normal encrypted traffic flows extracted from CTU-13 [30].

B. Experiment setup

Implementation. We implement D2LAD prototype using Keras [19] library. And we randomly sample 10% of our dataset to offline train the LSTM-based Autoencoder of our prototype.

Parameter setting. The parameter of D2LAD prototype is set as Table 1.

TABLE I
PARAMETER SETTING OF D2LAD PROTOTYPE.

Parameters	Values
n (Number of dictionary layers)	10
$ \mathbf{D}_i $ (Dimension of dictionary layer)	128
μ (Sparse parameter)	0.3
α (Weight of EWMA)	0.5
τ (Anomaly threshold)	0.95

Experiment environment. For our experiments, we run our prototype on a computer equipped with Intel(R) Xeon(R) CPU E5-2618L v3 CPU, 16GB of memory, NVIDIA GeForce RTX 2080 Ti GPU and 11GB of GPU memory from Matpool platform².

¹<https://tinyurl.com/tlsmalware>

²<https://www.matpool.com/>

C. Effectiveness evaluation

Metrics. Given raw traffic flows, our goal is to identify whether each flow is abnormal. We adopt the three most commonly used metrics of anomaly detection:

- (1) $Recall = \frac{TP(True\ positives)}{TP+FN(False\ negatives)}$;
- (2) $Precision = \frac{TP}{TP+FP(False\ positives)}$;
- (3) $F_1\ Score = 2 \times \frac{Recall \times Precision}{Recall + Precision}$.

Comparison. We compare our unsupervised method with two representative state-of-the-art methods: (1) *FS-Net* [13] (Supervised method) is an end-to-end framework that learns the labeled raw traffic to classify the traffic; (2) *Method of* [14] (Semi-supervised) firstly clusters the traffic and classify the clusters using labeled data.

Setup. First, to simulate the real-world condition of non-stationary and imbalanced traffic distribution, we adjust the flow number ratio of the malware dataset to the benign dataset according to 1:5, 1:10, and 1:20, through randomly duplicating flows from the benign dataset. Secondly, besides training the comparison methods using the original labels of datasets, we also add some noisy labels into the training set in order to make these methods tackle the noisy label problem [8], [15]. Specifically, according to [8], we randomly relabel 0.5%, 1%, and 5% samples. Moreover, we use 10-fold cross-validation to examine the methods.

Results. The experimental results are shown in Table 2. According to these results, we make the observation as follows.

- Our method reaches high accuracy ($F_1\ score \approx 94.5\%$) and outperforms the comparison methods when the ratio of malware dataset to benign dataset is relatively high (i.e., 1:10 and 1:20), which is suitable to the real-world imbalanced traffic distribution (The results of *Recall* for comparison methods are especially affected by this). It is because that the deep dictionary in our method is asymptotically affected by the samples and will be effectively converged to the normal patterns (we will demonstrate this in the next subsection). We can conclude that our method is robust to non-stationary traffic distribution.
- Our method is immune to the noisy label problem and achieves comparable accuracy to the comparison methods. Apparently, it is because our unsupervised method only depends on raw traffic data instead of labeled data. On the contrary, the accuracy comparison methods are very sensitive to the noisy labeled data.

D. Performance evaluation

Accuracy across data volume. We test our method for 20 times by inputting shuffled dataset and record the mean values and standard deviations of $F_1\ score$ with interval of 1,000 flows. Moreover, in order to explore the internal operation (normal pattern construction) of our method, we convert the multidimensional vectors of the dictionary layer ($\mathbf{D}_n$) to One-dimensional Word Vectors (OWV) using word2vec framework³. The results are reported in Fig. 3 (red line and green

³<https://github.com/danielfrg/word2vec>

TABLE II
EFFECTIVENESS COMPARISON.

Methods	Percentage of Noisy Label	Ratio of Malware Dataset to Benign Dataset								
		1:5			1:10			1:20		
		Recall	Precision	F_1 Score	Recall	Precision	F_1 Score	Recall	Precision	F_1 Score
FS-Net [13]	0%	95.63%	98.66%	97.12%	91.44%	95.21%	93.29%	87.53%	92.05%	89.73%
	0.5%	95.42%	98.26%	96.82%	89.14%	94.75%	91.86%	84.33%	91.13%	87.60%
	1%	95.31%	97.44%	96.36%	86.57%	93.87%	90.07%	81.16%	91.31%	85.94%
	5%	87.22%	90.17%	88.67%	81.44%	86.18%	83.74%	72.55%	84.72%	78.16%
Method of [14]	0%	96.11%	98.48%	97.28%	89.34%	97.22%	93.11%	84.34%	92.98%	88.45%
	0.5%	94.28%	97.42%	95.82%	87.52%	96.94%	91.98%	83.24%	92.10%	87.45%
	1%	93.73%	96.32%	95.01%	86.19%	95.90%	90.78%	77.13%	92.73%	84.21%
	5%	81.44%	79.82%	80.62%	72.25%	74.14%	73.18%	69.89%	75.48%	72.57%
Our Method	-	91.33%	95.65%	93.44%	92.87%	96.11%	94.46%	93.23%	96.32%	94.79%

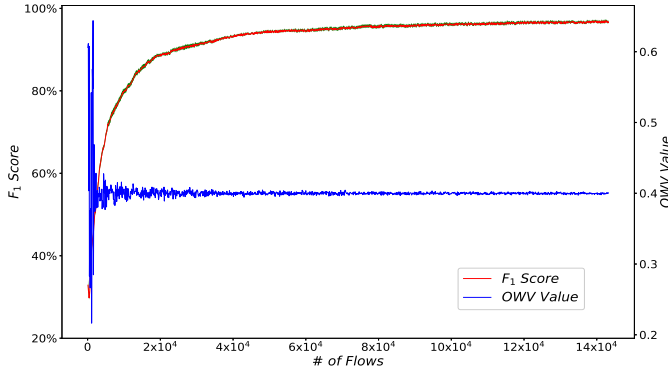


Fig. 3. F_1 Score and One-dimensional Word Vector (OWV) values across the number of flows.

shaded area for mean values standard deviations of F_1 score, blue line for OWV values). Based on the results, we can find that:

- We admit that our method faces a cold-start problem due to the random assignment for initial parameters of dictionary. However, the F_1 score raises steadily as data volume increases, and when around 2×10^4 flows were input, the F_1 score begins to converge to a value of 94.79%.
- The OWV value simultaneously begins to converge as the F_1 score. It can be inferred that the accuracy of our method depends on the internal construction and updating process of the deep dictionary.

Memory usage and time consumption. We measure the time consumption through recording the execution time of the thread for data processing in our prototype. We find that the mean and maximum processing rate of one flow is 187 ms and 353 ms. This relatively short processing time is because our method adopts a layer-wise dictionary parameter updating in a greedy manner. And apparently, it is needed only 1 flows in the memory to be processed each time. Therefore, our method achieves online detection.

V. RELATED WORK

In this section, we discuss existing works on detecting anomalies in encrypted traffic based on machine learning,

which can be generally classified into two categories: supervised methods and semi-supervised methods.

Supervised methods. The method of [9] is a supervised machine learning models that take advantage of a unique and diverse set of network flow data features. [8] analyzes six common machine learning algorithms, and show how they each performed on the problem of detecting malicious encrypted network sessions. [10] provides a comprehensive study of malware's use of TLS by observing the unencrypted TLS handshake messages and uses common machine learning to classify them. [11] distinguishes malicious encrypted HTTP traffic from benign encrypted traffic by applying three machine learning techniques. [12] adopts convolution neural network to extract features from encrypted traffic payloads. And [13] is an end-to-end classification model that learns features from raw encrypted traffic.

Semi-supervised methods. The authors of [14] propose a distance-based method that clusters unsupervised data as *FClass*, and build a *FClass* identification framework trained by XGBoost using labeled data.

These supervised and semi-supervised methods suffer from the problems of data of noisy label, non-stationary traffic distribution, and offline training with huge memory usage and time consumption. However, our unsupervised and online method is able to overcome the above problems.

VI. CONCLUSION

Detecting anomalies in encrypted traffic is a great challenge. In this paper, we presented D2LAD, which is a deep dictionary-based method. D2LAD aims at extracting sequential encrypted traffic features, using deep dictionary to represent the normal traffic pattern, and making use of the diverge of data to the dictionary to detect the anomalies. We implemented a D2LAD prototype, and thoroughly evaluated it. Our experiment results demonstrated that D2LAD achieves an accuracy of 94.5% and outperforms the state-of-the-art methods in real-world settings. In future work, we look forward to parallelizing the updating process of deep dictionary to enhance its performance.

ACKNOWLEDGMENT

This work is supported by the National Key Research and Development Program of China (2018YFB2100404,

2018YFB1800601), the Key Research and Development Program of Zhejiang Province (2018C03052, 2020C01021), and Major Scientific Project of Zhejiang Lab (2018FD0ZX01).

REFERENCES

- [1] A. Lakhina, M. Crovella, and C. Diot, "Mining anomalies using traffic feature distributions," in *Proceedings of the ACM SIGCOMM 2005 Conference on Applications, Technologies, Architectures, and Protocols for Computer Communications, Philadelphia, Pennsylvania, USA, August 22-26, 2005*, 2005, pp. 217–228. [Online]. Available: <https://doi.org/10.1145/1080091.1080118>
- [2] S. Garg, K. Kaur, N. Kumar, G. Kaddoum, A. Y. Zomaya, and R. Ranjan, "A hybrid deep learning-based model for anomaly detection in cloud datacenter networks," *IEEE Trans. Network and Service Management*, vol. 16, no. 3, pp. 924–935, 2019. [Online]. Available: <https://doi.org/10.1109/TNSM.2019.2927886>
- [3] Encrypted traffic analytics. [Online]. Available: <https://www.cisco.com/c/en/us/solutions/enterprise-networks/enterprise-network-security/eta.html/>
- [4] Https encryption on the web. [Online]. Available: <https://transparencyreport.google.com/https/overview?hl=en/>
- [5] T. Nelms, R. Perdisci, and M. Ahamad, "Execscent: Mining for new c&c domains in live networks with adaptive control protocol templates," in *Proceedings of the 22th USENIX Security Symposium, Washington, DC, USA, August 14-16, 2013*, 2013, pp. 589–604. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/paper/nelms>
- [6] J. Su, S. Chen, B. Han, C. Xu, and X. Wang, "A 60gbps DPI prototype based on memory-centric FPGA," in *Proceedings of the ACM SIGCOMM 2016 Conference, Florianopolis, Brazil, August 22-26, 2016*, 2016, pp. 627–628. [Online]. Available: <https://doi.org/10.1145/2934872.2959079>
- [7] W. Melo, P. L. Jr., R. Antonello, S. F. L. Fernandes, and D. F. H. Sadok, "On the performance of DPI signature matching with dynamic priority," in *IEEE Symposium on Computers and Communications, ISCC 2014, Funchal, Madeira, Portugal, June 23-26, 2014*, 2014, pp. 1–6. [Online]. Available: <https://doi.org/10.1109/ISCC.2014.6912553>
- [8] B. Anderson and D. A. McGrew, "Machine learning for encrypted malware traffic classification: Accounting for noisy labels and non-stationarity," in *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Halifax, NS, Canada, August 13 - 17, 2017*, 2017, pp. 1723–1732. [Online]. Available: <https://doi.org/10.1145/3097983.3098163>
- [9] B. Anderson, "Identifying encrypted malware traffic with contextual flow data," in *Proceedings of the 2016 ACM Workshop on Artificial Intelligence and Security, AISec@CCS 2016, Vienna, Austria, October 28, 2016*, 2016, pp. 35–46. [Online]. Available: <https://doi.org/10.1145/2996758.2996768>
- [10] B. Anderson, S. Paul, and D. A. McGrew, "Deciphering malware's use of TLS (without decryption)," *J. Computer Virology and Hacking Techniques*, vol. 14, no. 3, pp. 195–211, 2018. [Online]. Available: <https://doi.org/10.1007/s11416-017-0306-6>
- [11] A. S. Shekhawat, F. D. Troia, and M. Stamp, "Feature analysis of encrypted malicious traffic," *Expert Syst. Appl.*, vol. 125, pp. 130–141, 2019. [Online]. Available: <https://doi.org/10.1016/j.eswa.2019.01.064>
- [12] M. Lotfollahi, R. S. H. Zade, M. J. Siavoshani, and M. Saberian, "Deep packet: A novel approach for encrypted traffic classification using deep learning," *CoRR*, vol. abs/1709.02656, 2017. [Online]. Available: <http://arxiv.org/abs/1709.02656>
- [13] C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li, "Fs-net: A flow sequence network for encrypted traffic classification," in *2019 IEEE Conference on Computer Communications, INFOCOM 2019, Paris, France, April 29 - May 2, 2019*, 2019, pp. 1171–1179. [Online]. Available: <https://doi.org/10.1109/INFOCOM.2019.8737507>
- [14] J. Liu, Z. Tian, R. Zheng, and L. Liu, "A distance-based method for building an encrypted malware traffic identification framework," *IEEE Access*, vol. 7, pp. 100 014–100 028, 2019. [Online]. Available: <https://doi.org/10.1109/ACCESS.2019.2930717>
- [15] Z. Gao, L. Guo, C. Ma, X. Ma, K. Sun, H. Xiang, X. Zhu, H. Li, and X. Liu, "AMAD: adversarial multiscale anomaly detection on high-dimensional and time-evolving categorical data," *CoRR*, vol. abs/1907.06582, 2019. [Online]. Available: <http://arxiv.org/abs/1907.06582>
- [16] S. Tariyal, A. Majumdar, R. Singh, and M. Vatsa, "Greedy deep dictionary learning," *CoRR*, vol. abs/1602.00203, 2016. [Online]. Available: <http://arxiv.org/abs/1602.00203>
- [17] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997. [Online]. Available: <https://doi.org/10.1162/neco.1997.9.8.1735>
- [18] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, 2014*, pp. 1724–1734. [Online]. Available: <https://www.aclweb.org/anthology/D14-1179/>
- [19] F. Chollet. Keras. [Online]. Available: <https://github.com/fchollet/keras/>
- [20] M. Chen, T. Huang, V. Y. Shu, C. Chen, T. Hsieh, and N. Y. Yen, "Learning the chinese sentence representation with LSTM autoencoder," in *Companion of the The Web Conference 2018 on The Web Conference 2018, WWW 2018, Lyon, France, April 23-27, 2018*, 2018, pp. 403–408. [Online]. Available: <https://doi.org/10.1145/3184558.3186355>
- [21] F. Zhao, J. Feng, J. Zhao, W. Yang, and S. Yan, "Robust lstm-autoencoders for face de-occlusion in the wild," *IEEE Trans. Image Processing*, vol. 27, no. 2, pp. 778–790, 2018. [Online]. Available: <https://doi.org/10.1109/TIP.2017.2771408>
- [22] T. Fu, T. N. Hoang, C. Xiao, and J. Sun, "DDL: deep dictionary learning for predictive phenotyping," in *Proceedings of the Twenty-Eighth International Joint Conference on Artificial Intelligence, IJCAI 2019, Macao, China, August 10-16, 2019*, 2019, pp. 5857–5863. [Online]. Available: <https://doi.org/10.24963/ijcai.2019/812>
- [23] A. Shiravi, H. Shiravi, M. Tavallaee, and A. A. Ghorbani, "Toward developing a systematic approach to generate benchmark datasets for intrusion detection," *Computers & Security*, vol. 31, no. 3, pp. 357–374, 2012. [Online]. Available: <https://doi.org/10.1016/j.cose.2011.12.012>
- [24] A. Makhzani and B. J. Frey, "k-sparse autoencoders," in *2nd International Conference on Learning Representations, ICLR 2014, Banff, AB, Canada, April 14-16, 2014, Conference Track Proceedings*, 2014. [Online]. Available: <http://arxiv.org/abs/1312.5663>
- [25] I. Daubechies, M. Debrise, and C. De Mol, "An iterative thresholding algorithm for linear inverse problems with a sparsity constraint," *Communications on Pure and Applied Mathematics: A Journal Issued by the Courant Institute of Mathematical Sciences*, vol. 57, no. 11, pp. 1413–1457, 2004.
- [26] X. Hou and L. Zhang, "Dynamic visual attention: searching for coding length increments," in *Advances in Neural Information Processing Systems 21, Proceedings of the Twenty-Second Annual Conference on Neural Information Processing Systems*, 2008, pp. 681–688.
- [27] Malware traffic analysis. [Online]. Available: <https://www.malware-trafficanalysis.net/>
- [28] Stratosphere ips. malware capture facility project. [Online]. Available: <https://www.stratosphereips.org/datasets-malware>
- [29] Lastline. [Online]. Available: <https://www.lastline.com/>
- [30] Ctu-13 dataset. [Online]. Available: <https://www.stratosphereips.org/datasets-ctu13/>